

Проект по Софтверска Дефинирана Безбедност:

Music Playlist application with OPA - Open Policy Agent

Ментори:

Проф. д-р Ристе Стојанов

Асис. Дарко Сасански

Студент:

Васил Блажевски

211286

1. Општ опис на проектот

Проектот **Music Playlist Application with OPA** претставува веб-апликација за управување со музички плејлисти, развиена со Spring Boot. Апликацијата користи Spring Security за автентикација и заштита на HTTP ресурсите, додека одлуките за авторизација за операции над плејлистите се делегираат на **Open Policy Agent (OPA)**.

Главната цел на проектот е да се прикаже концептот на **policy-based authorization** и раздвојување на безбедносната политика од апликациската логика. Наместо правилата за пристап да бидат директно имплементирани во Java кодот, тие се дефинираат како надворешна политика во **Rego** јазикот и се извршуваат од OPA.

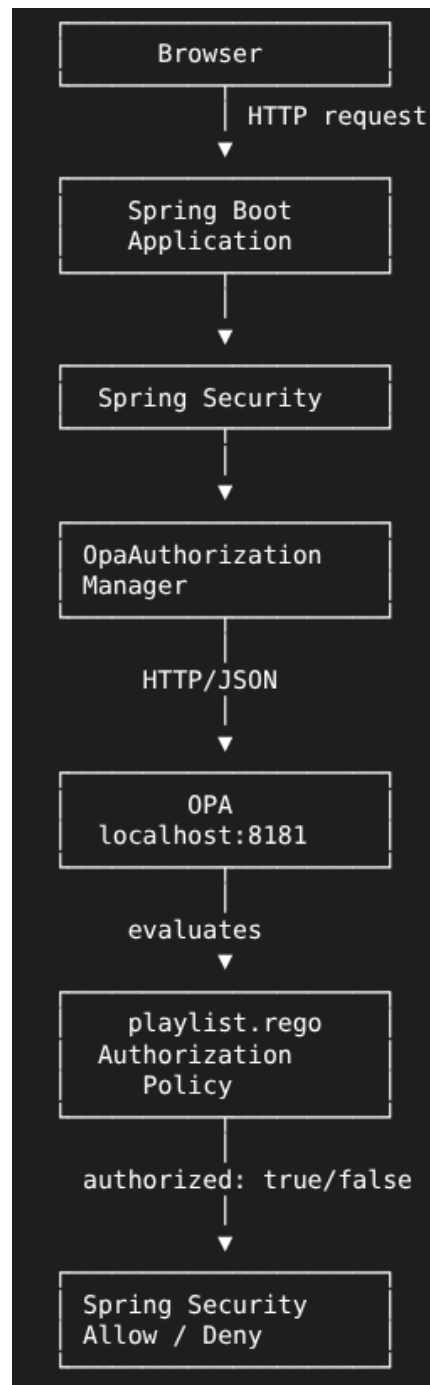
На овој начин, Spring Boot апликацијата не ја содржи целата логика за одредување дали одреден корисник смее да изврши дадена операција. Апликацијата испраќа структурирани информации до OPA, како што се корисникот, неговите привилегии, HTTP методот и URI-то на барањето. OPA ги споредува овие информации со дефинираната полиса и враќа одлука дали пристапот е дозволен или одбиен.

Проектот претставува работен прототип на ваков пристап и демонстрира како надворешен policy engine може да се интегрира со Spring Security.

2. Технологии користени

Technology	Purpose
Java 21	Application programming language
Spring Boot	Application framework
Spring MVC	Web application
Spring Security	Authentication and security enforcement
H2	Database
Thymeleaf	Web UI
Open Policy Agent	External authorization engine
Rego	Authorization policy language
JUnit / MockMvc	Integration testing

3. Архитектура на системот



Слика 1. Архитектура и тек на авторизација процесот

Дијаграмот го прикажува текот на едно HTTP барање од корисникот до конечната authorization одлука.

Процесот се одвива во следните чекори:

1. Browser → Spring Boot Application

Корисникот преку веб-прелистувачот испраќа HTTP барање кон Spring Boot апликацијата. Барањето може да биде, на пример, креирање, промена или бришење на музичка плејлиста.

2. Spring Boot Application → Spring Security

Барањето прво поминува низ Spring Security. Во оваа фаза се проверува безбедносниот контекст на корисникот, односно дали корисникот е автентизиран и кои авторитети/улоги ги поседува. Во проектот, Spring Security е конфигуриран така што одредени операции над плејлистите се проследуваат до OpaAuthorizationManager. На пример, операциите за креирање, зачувување, изменување и бришење на плејлисти се заштитени преку Opa authorization manager.

3. Spring Security → OpaAuthorizationManager

Кога пристигнува заштитено барање, Spring Security го повикува OpaAuthorizationManager, кој ја презема информацијата за моменталниот корисник и HTTP барањето. Од нив се формира JSON структура која содржи информации како:

username/principal;

authorities на корисникот;

URI на барањето;

HTTP метод;

Accept header.

Оваа логика е имплементирана директно во OpaAuthorizationManager

4. OpaAuthorizationManager → OPA

Потоа OpaAuthorizationManager испраќа HTTP POST барање до OPA. Во конфигурацијата на проектот OPA се очекува да работи на: **localhost:8181**, а policy endpoint-от е: **/v1/data/auth/playlist**. Барањето кон OPA користи JSON формат.

5. OPA → Rego policy

OPA го прима JSON input-от и го евалуира во однос на authorization policy напишана во Rego. Во овој проект полисата има задача да одреди дали конкретниот корисник има право да ја изврши бараната операција. Концептуално, OPA добива информации од типот:

```
{  
  "input": {  
    "principal": "user",  
    "authorities": ["ROLE_USER"],  
    "uri": "/music-playlists/1/edit",  
    "method": "GET"  
  }  
}
```

Потоа Rego полисата го обработува овој input и генерира authorization decision.

6. OPA → Spring Security

OPA враќа JSON response во кој се содржи authorization резултат. Во имплементацијата се чита полето:

```
result.authorized
```

Ако вредноста е true, барањето е дозволено. Ако е false, барањето е одбиено.

7. Spring Security → Allow / Deny

Конечната одлука се враќа во Spring Security преку AuthorizationResult.

authorized = true → Spring Security го дозволува барањето и апликацијата продолжува со извршување.

authorized = false → пристапот се одбива.

Во проектот е конфигурирана и посебна страница /access_denied која се користи при одбиен пристап.

4. Автентикација vs Авторизација

Во системот треба да се разликуваат два безбедносни концепти:

Автентикација (Authentication) одредува кој е корисникот. Во проектот ова се реализира преку Spring Security и UserDetailsService, кој ги користи корисничките податоци од базата.

Авторизација (Authorization) одредува што смее корисникот да направи откако е автентизиран.

Во овој проект, за операциите поврзани со создавање, менување или бришење на плејлиста или песна authorization одлуката не се носи директно во Spring Boot апликацијата, туку се делегира на OPA кои ни дава подобра авторизациска логика.

5. Интеграција со OPA

За интеграција со Open Agency Policy е имплементирана посебна Java компонента **ОpaAuthorizationManager**, која го имплементира Spring Security AuthorizationManager<RequestAuthorizationContext> интерфејсот.

Кога Spring Security ќе побара authorization проверка, OpaAuthorizationManager:

1. го зема тековниот Authentication;
2. го зема HTTP request-от;
3. ги извлекува principal-от и authorities;
4. го зема URI-то и HTTP методот;

5. креира JSON payload;
6. испраќа HTTP POST до OPA;
7. го чита authorized резултатот;
8. враќа AuthorizationResult до Spring Security.

Конфигурацијата за OPA е издвоена во `OpaAuthorizationProperties`, каде што се дефинирани `base URL` и `policy path`. Ова е важно бидејќи OPA не е директно вграден во `application logic`-от. Наместо тоа, Spring Boot апликацијата комуницира со него преку HTTP/JSON интерфејс.

6. Зошто ова е Software-Defined Security?

Проектот е релевантен за Software-Defined Security бидејќи безбедносните правила се дефинираат како софтверска политика, наместо да бидат статички вградени во апликацискиот код. Во традиционален пристап, `authorization` правилата можат да бидат директно имплементирани во Java кодот:

```
if (user.isAdmin()) {  
    allow();  
}
```

Во пристапот со OPA, правилата се издвојуваат во посебна `policy layer`. Апликацијата му испраќа контекст на OPA, а OPA ја носи `authorization` одлуката.

Главните предности се:

- **Separation of concerns** – апликациската логика е одделена од `security policy`;
- **Policy as Code** – безбедносните правила се чуваат и управуваат како код;
- **Централизирана авторизација** – повеќе апликации можат да користат ист `policy engine`;
- **Полесна промена на правила** – `authorization` правилата може да се менуваат без промена на главната Java логика;
- **Конзистентност** – истите правила можат да се применуваат на различни ресурси и сервиси.

Ова го прави OPA особено погоден за системи каде што безбедносните политики се менуваат почесто од самата апликациска логика.

7. Rego полиси

Во проектот се користи Open Policy Agent (OPA) и јазикот Rego за дефинирање на правилата за авторизација поврзани со музичките плејлисти. Полисата е сместена во пакетот `auth.playlist` и одредува дали дадено барање е дозволено врз основа на HTTP методот, URI-то, корисникот и неговите овластувања. Полисата започнува со:

```
package auth.playlist
default authorized := false
```

Пакетот `auth.playlist` ги групира сите правила поврзани со пристапот до функционалностите за плејлисти. `authorized` има почетна вредност **false**. Ова претставува **deny-by-default** пристап, односно секое барање е одбиено доколку не постои конкретно правило што го дозволува.

1. Одлука за авторизација

```
authorized if {
  count(allow) > 0
}
```

Правилото проверува дали постои барем едно успешно правило во `allow`.

Секое правило што го исполнува условот додава дозвола во **allow**. Ако ниту едно правило не се совпадне со барањето, **allow** останува **празен** и

authorized останува **false**. На овој начин се спречува случајно дозволување на пристап кога не е дефинирано соодветно правило.

2. Креирање плејлиста полиса

За пристап до формата за креирање плејлиста е потребно корисникот да биде најавен:

```
allow["playlist_create_form"] if {  
  input.method == "GET"  
  input.uri == "/music-playlists/create"  
  is_authenticated  
}
```

Правилото бара:

HTTP методот да биде GET;
URI-то да биде /music-playlists/create;
корисникот да биде автентизиран.
Посебно правило го дозволува самото креирање на плејлистата:

```
allow["playlist_create"] if {  
  input.method == "POST"  
  input.uri == "/music-playlists/save"  
  is_authenticated  
}
```

Овде се користи POST метод и URI /music-playlists/save. Со тоа се прави разлика помеѓу **отворање** на формата и **испраќање** на податоците за креирање плејлиста.

3. Ажурирање плејлиста полиса

Автентизиран корисник може да ја отвори страницата за Ажурирање(editing):

```
allow["playlist_edit"] if {  
  input.method == "GET"  
  regex.match("^/music-playlists/[^/]+/edit$", input.uri)
```

```
    is_authenticated
}
```

Тука се користи регуларен израз за да се препознаат URI адреси како:

```
/music-playlists/1/edit
/music-playlists/25/edit
```

Делот `[^/]+` го претставува ID-то на плејлистата.

4.3а **зачувување на промените** се користи посебно правило:

```
allow["playlist_update"] if {
    input.method == "POST"
    regex.match("^/music-playlists/[^/]+/update$", input.uri)
    is_authenticated
}
```

Ова значи дека барањето мора да биде POST и да има соодветна `/update` патека.

5. **Бришење плејлиста** е дефинирано со:

```
allow["playlist_delete"] if {
    input.method == "POST"
    regex.match("^/music-playlists/[^/]+/delete$", input.uri)
    is_authenticated
}
```

И тука е потребен автентизиран корисник, а операцијата мора да се изврши преку POST барање.

6. **Администраторски пристап** имаат дополнително правило:

```
allow["admin"] if {
    has_authority("ROLE_ADMIN")
}
```

Доколку корисникот ја има улогата `ROLE_ADMIN`, правилото го смета корисникот за авторизиран. Ова овозможува администраторите да имаат поширок пристап од обичните корисници.

7.Проверка дали корисникот е најавен

Автентикацијата се проверува преку:

```
is_authenticated if {  
    input.principal != "anonymousUser"  
}
```

Доколку `principal` не е **`anonymousUser`**, корисникот се смета за **автентизиран**. Со други зборови, `anonymousUser` претставува корисник кој нема активна автентизирана сесија.

8.Проверка на кориснички овластувања

Проверката за одредена улога или `authority` е дефинирана со:

```
has_authority(required) if {  
    some i  
    input.authorities[i] == required  
}
```

Ова правило ги проверува сите овластувања на корисникот и проверува дали меѓу нив постои бараната вредност.

На пример, повикот:

```
has_authority("ROLE_ADMIN")
```

ќе биде успешен доколку `ROLE_ADMIN` постои во `input.authorities`.

8. Тестирање

Класата **`MusicPlaylistSecurityTest`** проверува дали правилата за безбедност и авторизација функционираат правилно во рамките на Spring апликацијата. За тестирање се користи:

@SpringBootTest го стартува Spring контекстот
@AutoConfigureMockMvc овозможува испраќање HTTP барања преку MockMvc, без потреба од стартување на вистински веб-сервер

За симулирање на најавен корисник се користи:

.with(user("user-42").roles("USER"))

За POST барањата се користи и:

.with(csrf())

со цел да се задоволи Spring Security CSRF заштитата.

Тестови:

1. authenticatedUser_canOpenCreatePlaylistForm:

Најавен корисник може да ја отвори формата за креирање
испраќа GET барање кон:

/music-playlists/create

како најавен корисник со улога USER.

Се очекува:

status().isOk()

односно HTTP статус 200 OK.

Овој тест потврдува дека автентифицираните корисници можат да пристапат
до формата за креирање плејлиста.

2. authenticatedUser_canCreatePlaylist:

Најавен корисник може да креира плејлиста

испраќа POST барање до:

/music-playlists/save

Корисникот е најавен и му се испраќаат параметри како име, опис и ID на
песна.

Очекуваниот резултат е 3xx статус, односно успешно извршување
проследено со redirect.

Со овој тест се проверува дека автентифициран корисник може да креира
нова плејлиста.

3. playlistOwner_canOpenEditPage:

Корисникот може да ја отвори страницата за уредување

Тестот ја пронаоѓа плејлистата "Chill Night Drive" и испраќа GET барање до: **/music-playlists/{id}/edit**

како автентичиран корисник. Се очекува:

`status().isOk()`

што покажува дека пристапот до страницата за уредување е дозволен.

4.differentUser_cannotUpdatePlaylist:

Друг корисник не може да ја ажурира плејлистата

Тестот се обидува да ја ажурира плејлистата преку корисникот:

`someone-else`

Наместо дозвола, се очекува:

`status().isForbidden()`

односно HTTP 403 Forbidden.

Овој тест е особено важен бидејќи покажува дека самото најавување не значи дека корисникот може да менува туѓи ресурси.

Апликацијата дополнително ја проверува сопственоста на плејлистата.

5.differentUser_cannotDeletePlaylist:

Друг корисник не може да ја избрише плејлистата

Тестот проверува дали друг корисник може да испрати барање за бришење на туѓа плејлиста.

Очекуваниот резултат е: 403 Forbidden и со ова се потврдува дека заштитата од неовластено бришење функционира.

6.anonymousUser_cannotOpenCreatePlaylistForm:

Анонимен корисник не може да ја отвори формата

Тестот испраќа GET барање без да се дефинира корисник.

Очекуваниот резултат е 3xx redirect.

Ова е очекувано однесување бидејќи Spring Security го пренасочува неавтентичираниот корисник кон login процесот.

7.anonymousUser_cannotCreatePlaylist Анонимен корисник не може да креира плејлиста

Тестот испраќа POST барање кон `/music-playlists/save` без најавен корисник.

И тука се очекува 3xx redirect.

Со ова се потврдува дека креирањето плејлиста е достапно само за автентичирани корисници.

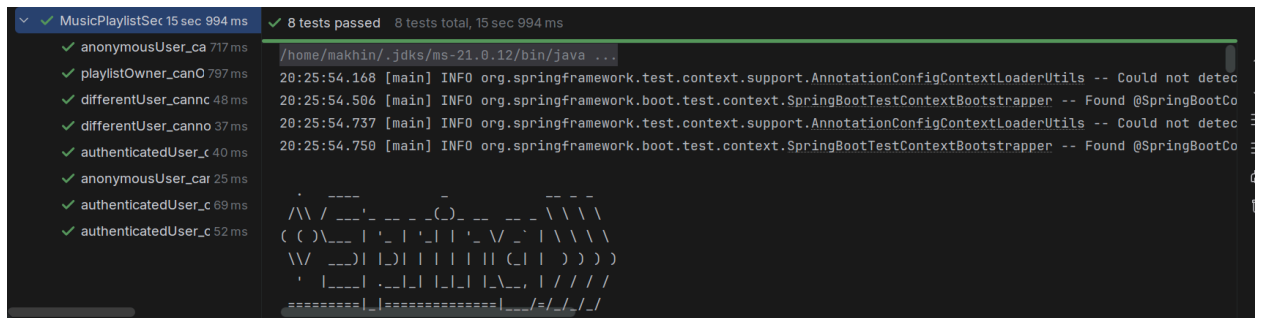
8. authenticatedUser_cannotUseWrongHttpMethod:

Автентизиран корисник не може да користи погрешен HTTP метод
Последниот тест испраќа GET барање кон:

/music-playlists/{id}/delete

Иако корисникот е најавен, Rego политиката за бришење бара:
POST, а не GET. Затоа очекуваниот резултат е: 403 Forbidden

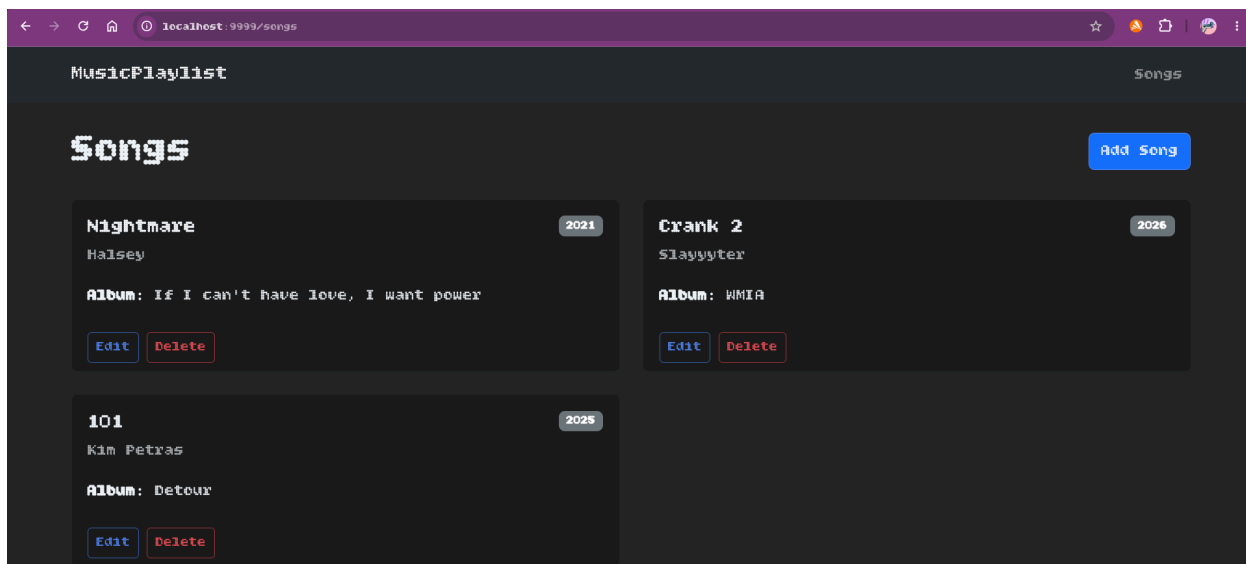
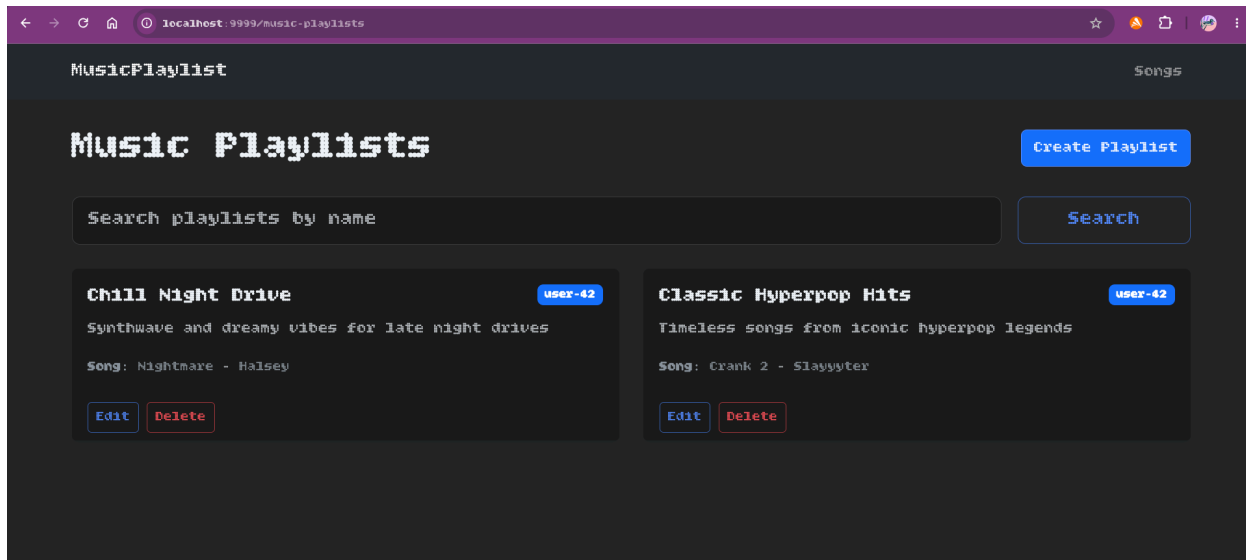
Овој тест потврдува дека авторизацијата не зависи само од URI-то, туку и
од HTTP методот.



Слика 2. Приказ на функционалност на тестови

9. Практична демонстрација на OPA

При пристап на страната <http://localhost:9999/music-playlists> се гледат
предефинираните плејлисти и на <http://localhost:9999/songs> се прикажуваат
песните:



При обид на кликање на било кое копче на страната, како на пример **Create Playlist** или **Add Song** корисникот се пренасочува кон страницата за најава, ова е контролирано од **OpaAuthorizationManager** во **SecurityConfig**:

← → ↻ 🏠 ⓘ localhost:9999/login

Sign in

Username

Password

[Admin area](#) [Don't have an account? Register here!](#)

Ова прикажува неавтентициран корисник нема право на никаква акција со плејлиста или песна според полиса. Ако нема профил, тогаш корисникот прво си праве профил па се најавува и се пренасочува кон страницата за најава па кон страницата за плејлисти:

← → ↻ 🏠 ⓘ localhost:9999/register

Register

Username

Password

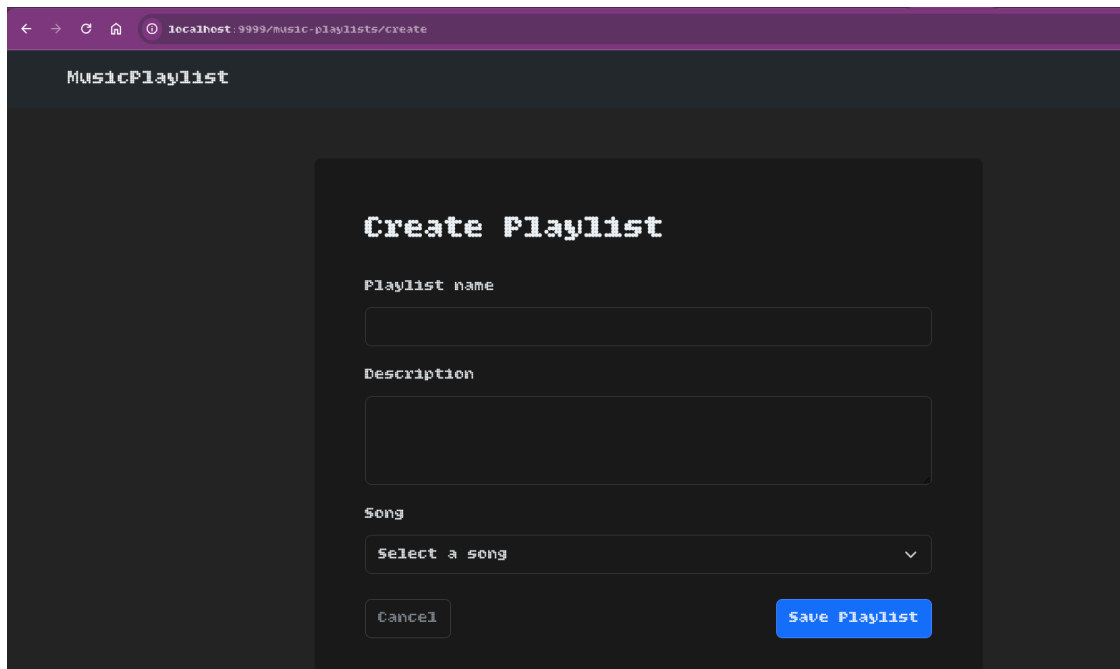
Repeat Password

Name

Surname

[Already have an account? Login here!](#)

Следно ќе се прикаже создавање на нова плејлиста при клик на **Create Playlist** копчето дозволено од полисата **playlist_create_form**:



The screenshot shows a web browser window with the address bar displaying 'localhost:9999/music-playlists/create'. The page title is 'MusicPlaylist'. The main content area features a dark-themed form titled 'Create Playlist'. The form includes three input fields: 'Playlist name', 'Description', and 'Song'. The 'Song' field is a dropdown menu with the placeholder text 'select a song'. At the bottom of the form, there are two buttons: 'Cancel' and 'Save Playlist'.

MusicPlaylist

Create Playlist

Playlist name

Description

Song

select a song

Cancel Save Playlist

The screenshot shows a web browser window with the address bar displaying 'localhost:9999/music-playlists/create'. The page has a dark theme and a header with the text 'MusicPlaylist'. The main content area is a modal form titled 'Create Playlist' in a pixelated font. The form contains three input fields: 'Playlist name' with the value '101', 'Description' with the value '11010101010101', and 'Song' with a dropdown menu showing '101 - Kim Petras'. At the bottom of the form are two buttons: 'Cancel' and 'Save Playlist'.

localhost:9999/music-playlists/create

MusicPlaylist

Create Playlist

Playlist name

101

Description

11010101010101

Song

101 - Kim Petras

Cancel Save Playlist

Со клик на **Save Playlist** се создава новиот пополнет плејлиста, дозволено од полисата `playlist_create`:

The screenshot shows a web browser window with the address bar displaying 'localhost:9999/music-playlists'. The page has a dark theme and a header with the text 'MusicPlaylist' and a 'Songs' link. The main content area is titled 'Music Playlists' in a pixelated font. There is a search bar with the placeholder text 'Search playlists by name' and a 'Search' button. A 'Create Playlist' button is in the top right corner. Below the search bar, there are three playlist cards. Each card displays the playlist name, a description, the song name, and 'Edit' and 'Delete' buttons. The first two cards are for 'Chill Night Drive' and 'Classic Hyperpop Hits', both created by 'user-42'. The third card is for '101', created by 'Vasil', and matches the data from the 'Create Playlist' form.

localhost:9999/music-playlists

MusicPlaylist Songs

Music Playlists

Create Playlist

Search playlists by name Search

Chill Night Drive user-42

Synthwave and dreamy vibes for late night drives

Song: Nightmare - Halsey

Edit Delete

Classic Hyperpop Hits user-42

Timeless songs from iconic hyperpop legends

Song: Crank 2 - Slayyyter

Edit Delete

101 Vasil

11010101010101

Song: 101 - Kim Petras

Edit Delete

Процесот за создавање нова песна е ист како за плејлиста:

localhost:9999/songs/create

Playlist

Add Song

Title
Runaway

Artist
Avril Lavigne

Album
The Best Damn Thing

Year
2007

Cancel Save Song

localhost:9999/songs

MusicPlaylist

Songs

Songs

Add Song

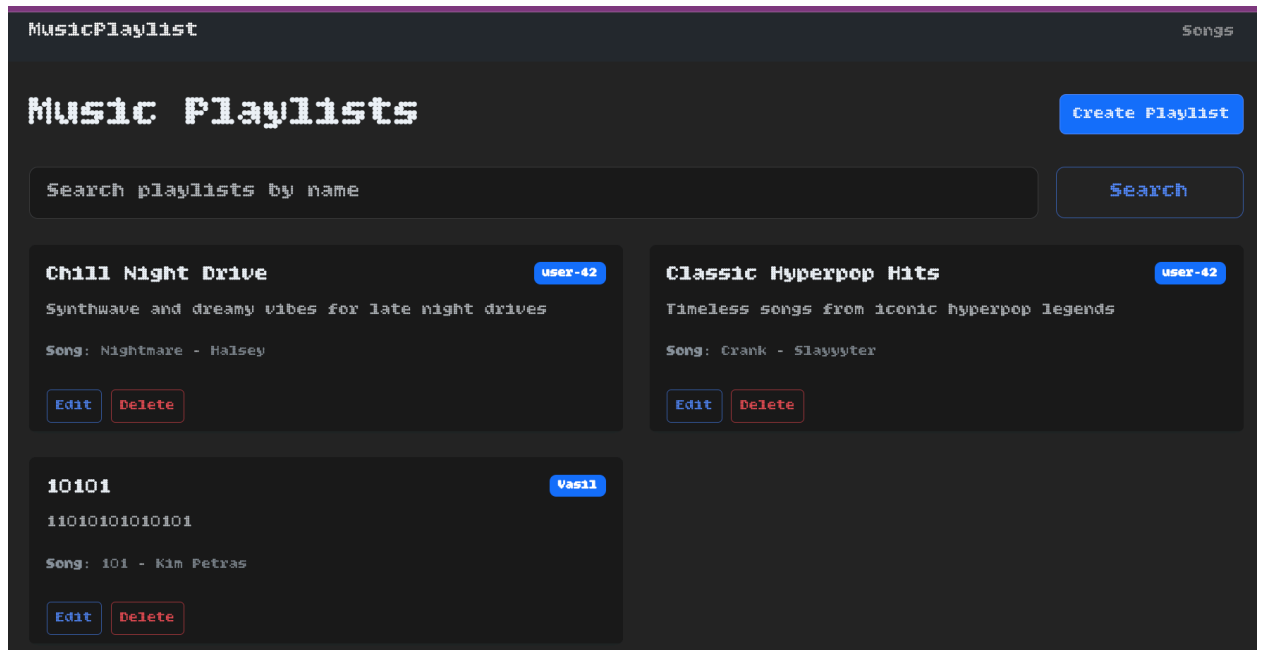
Nightmare 2021
Halsey
Album: If I can't have love, I want power
Edit Delete

Crank 2 2026
Slayyyter
Album: WMIA
Edit Delete

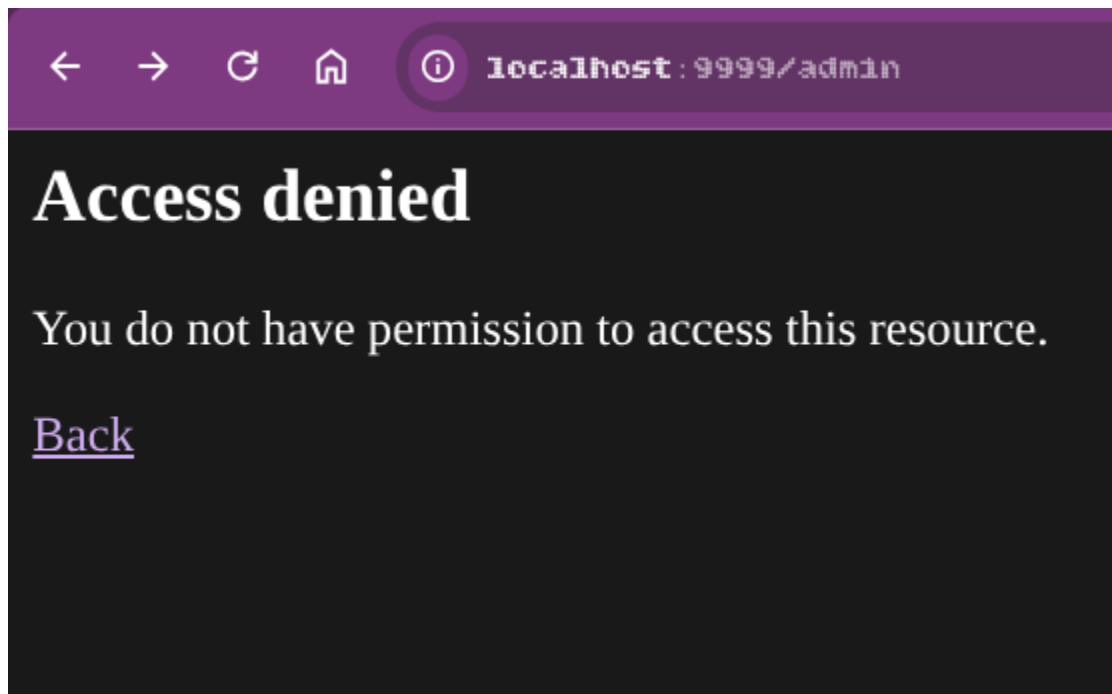
101 2025
Kim Petras
Album: Detour
Edit Delete

Runaway 2007
Avril Lavigne
Album: The Best Damn Thing
Edit Delete

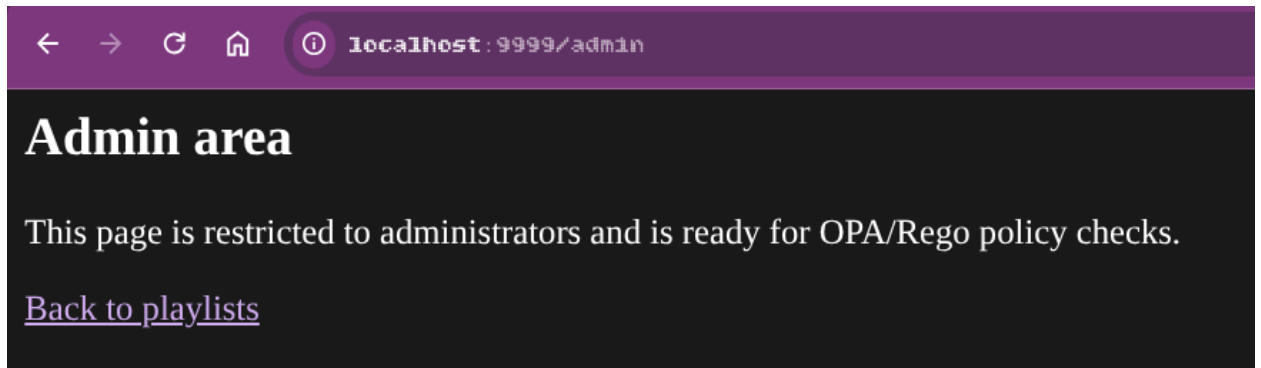
Следно се прикажува полисата **playlist_edit** при клик на копчето edit на плејлиста и успешно ажурирање со полисата **playlist_update** на плејлистата 101 во 10101:



За крај се покажува полисата **admin**, во моментот сум најавен како user и при обид на пристап кон <http://localhost:9999/admin> добивав:



Но, со пристап кон <http://localhost:9999/logout> се најавувам со admin улога и новиот приказ кон <http://localhost:9999/admin> ми е:



Со дадена дозвола на admin area.

10. Што научивме

Преку имплементацијата на прототипот беше демонстрирано како Open Policy Agent може да се интегрира со Spring Security за донесување на authorization одлуки.

Најважниот концепт што е демонстриран е раздвојувањето на policy decision од application logic. Spring Boot апликацијата е одговорна за обработка на барањето и негово спроведување, додека OPA е одговорен за евалуација на безбедносната политика.

Со користење на Rego, authorization правилата можат да се менуваат и одржуваат независно од Java кодот. Овој пристап овозможува поголема флексибилност и претставува практична примена на policy-based и software-defined security.

Прототипот исто така покажа дека интеграцијата може да се реализира преку едноставен HTTP/JSON интерфејс, при што OPA функционира како независен authorization service

Линк до проект: <https://github.com/Vasilxb/MusicPlaylistOPA.git>